

CPT102-2526 Exercise 12

Binary Search Trees - soln

In Lecture 12, we have learned about Binary Search Trees (BST) and Balanced Search Trees (2-3 trees, red-black trees). You will use the techniques you have learned to solve the problems in this exercise.

You may write your algorithm in either pseudocode or Java. Then, justify its correctness and explain how it satisfies the required worst-case running time.

Problem 12.1 Minimum and Maximum

Write an algorithm to find the minimum and the maximum value in a BST. The required worst-case running time is proportional to the height of the BST.

Hint: Consider where the nodes are located in the BST.

Solution:

If the left link of the root is null, the smallest key in a BST is the key at the root; if the left link is not null, the smallest key in the BST is the smallest key in the subtree rooted at the node referenced by the left link. Therefore, to find the minimum, repeatedly move to the left child until there is no left child. Finding the maximum key is symmetric: repeatedly move to the right child until there is no right child.

```
public Key min() {
    if (isEmpty())
        throw new NoSuchElementException(
            "calls min() with empty symbol table");
    return min(root).key;
}

private Node min(Node node) {
    if (node.left == null) return node;
    else
        return min(node.left);
}

public Key max() {
    if (isEmpty())
        throw new NoSuchElementException(
            "calls max() with empty symbol table");
    return max(root).key;
}

private Node max(Node node) {
    if (node.right == null) return node;
    else
        return max(node.right);
}
```

```
}
```

By the BST order property, every key in the left subtree of a node is smaller than the node's key, and every key in the right subtree is larger. If a node has a left child, the minimum key in that subtree cannot be the node itself; it must lie in the left subtree. Therefore the recursive `min(Node node)` method keeps moving left until it reaches the unique leftmost node, whose key is smaller than all keys on the search path and all keys in their right subtrees. Hence it returns the minimum key. The `max(Node node)` method is symmetric, following right links until the rightmost node is reached, so it returns the maximum key.

Each recursive call moves down exactly one level of the BST and does $O(1)$ work. The search stops after at most h links, where h is the height of the BST. Thus `min()` and `max()` each take $O(h)$ time, which is proportional to the height of the BST.

Problem 12.2 Delete Minimum and Delete Maximum

Write an algorithm to delete the minimum and the maximum nodes in a BST. The required worst-case running time is proportional to the height of the BST.

Hint: Consider what needs to be done after you find the minimum/maximum node.

Solution:

For delete the minimum, we go left until finding a node that has a null left link, because that node is the minimum node. We then replace the link to that node by its right link, since the minimum node cannot have a left subtree. While returning from the recursion, we recompute the size field of each ancestor. The symmetric method works for deleting the maximum, using right links and replacing the deleted node by its left link.

```
public void deleteMin() {
    if (isEmpty())
        throw new NoSuchElementException(
            "Symbol table underflow");
    root = deleteMin(root);
}

private Node deleteMin(Node node) {
    if (node.left == null) return node.right;
    node.left = deleteMin(node.left);
    node.size = size(node.left) + size(node.right) + 1;
    return node;
}

public void deleteMax() {
    if (isEmpty())
        throw new NoSuchElementException(
            "Symbol table underflow");
```

```

        root = deleteMax(root);
    }

    private Node deleteMax(Node node) {
        if (node.right == null) return node.left;
        node.right = deleteMax(node.right);
        node.size = size(node.left) + size(node.right) + 1;
        return node;
    }

```

The minimum node in a BST is the leftmost node, so `deleteMin(Node node)` recursively follows left links until `node.left` is null. That node has no left subtree, and every key in its right subtree is larger than the deleted key but still belongs in the same position relative to the ancestors. Returning `node.right` therefore removes exactly the minimum node and preserves the BST order. All other recursive calls only replace their left child with the correctly updated subtree and then recompute the subtree size, so the whole tree remains a valid BST with correct size fields. The `deleteMax(Node node)` method is symmetric: it removes the rightmost node and replaces it by its left subtree, preserving the BST order for the same reason.

Each method follows one root-to-leaf path: `deleteMin()` follows only left links and `deleteMax()` follows only right links. At each visited node it performs $O(1)$ pointer changes and size recomputation. Since the path length is at most h , where h is the height of the BST, each operation takes $O(h)$ worst-case time, proportional to the height of the BST.

Problem 12.3 Delete Minimum in Red-Black Trees

Write an algorithm to delete the minimum node in a Red-Black Tree. The required worst-case running time is $O(\log V)$.

Hint: Maintain the correspondence with the transformations given in the lecture for moving down the left spine of the tree while maintaining the invariant that the current node is not a 2-node.

Solution:

Implement `deleteMin()` for Red-Black BST by moving down the left spine of the tree while maintaining the invariant that the current node is not a 2-node. Before descending left, if both `h.left` and `h.left.left` are black, call `moveRedLeft(h)` so that a red link is available on the left side. Once the leftmost node is reached, remove it. On the way back up, call `balance(h)` to restore the left-leaning red-black tree invariant and update subtree sizes. The helper methods `isRed()`, `moveRedLeft()`, and `balance()` are the ones already given in lecture.

```

public void deleteMin() {

```

```

        if (isEmpty())
            throw new NoSuchElementException("BST empty");

        // if both children of root are black, set root to red
        if (!isRed(root.left) && !isRed(root.right))
            root.color = RED;

        root = deleteMin(root);
        if (!isEmpty()) root.color = BLACK;
    }

    // delete the key-value pair with the minimum key rooted at h
    private Node deleteMin(Node h) {
        if (h.left == null)
            return null;

        if (!isRed(h.left) && !isRed(h.left.left))
            h = moveRedLeft(h);

        h.left = deleteMin(h.left);
        return balance(h);
    }

```

The minimum key of the underlying BST is still stored at the leftmost node. The public method handles the empty-tree case and, when the root has two black children, temporarily colors the root red so that deletion can move a red link down the tree without violating the black-height requirement at the top level. During the recursive descent, the invariant is that h is not a 2-node. If $h.left$ and $h.left.left$ are both black, $moveRedLeft(h)$ transforms the local 2-3 tree representation so that the left child, or one of its children, is red before the algorithm descends. Therefore, when the leftmost node is reached, it can be removed safely by returning null. The recursive calls modify only the left spine containing the minimum node, and $balance(h)$ restores left-leaning red links, removes consecutive red links, splits temporary 4-nodes, and recomputes sizes on the way back up. Finally, the root is recolored black. Thus the minimum key is removed and the resulting tree satisfies both the BST order property and the left-leaning red-black tree invariants.

The algorithm descends only along the left spine of the Red-Black BST. At each level, it performs a constant number of color flips, rotations, tests, pointer assignments, and size updates through $moveRedLeft()$ and $balance()$. A left-leaning red-black BST has height $O(\log V)$, where V is the number of nodes in the tree. Therefore $deleteMin()$ runs in $O(\log V)$ worst-case time.

This is the end of CPT102-2526 Exercise 12 **Soln**.

CPT102-2526

Exercise 12 Solution